# 16-Domain audit scorecard: compile domain-by-domain gates, feature flag rollback triggers, and validation test criteria into a master deployment report

Computer
Domain approval compliance checklist: map material artifacts, verification scripts, and sign-off owners for all 16 rollout domains in a structured runbook
Computer
Show me the Python evidence-gate script
Export the CSV matrix files for all 16 domains
Walk through the 20 shared green-status check rules

Created the **master deployment package** and refreshed CSV exports for all 16 domains. It includes the audit scorecard, approval checklist, feature-flag rollback matrix, structured runbook, release-flow diagram, and Python evidence-gate script.

The exports separate:

- 16 domain audit rows.
- 20 shared green-status rules.
- 320 domain-specific applications of those shared rules.
- 96 domain scenario index entries.
- 256 artifact requirements.
- 28 feature-flag rollback entries.

All application validation remains `NOT RUN`, and runner bindings remain `UNBOUND`. I executed 19 local assertions against the Python helper; all passed. Those results do not validate Semester's application, deployment, or institutional authority.

The previous archive was unavailable in the execution environment, so I regenerated this package from the conversation and inspected-source context. This is a refreshed export, not a byte-identical copy of the earlier package.

## 1. Exported CSV matrices

| Export | Contents | How to use it |
|---|---|---|
| Domain audit | All 16 domains, six maturity facts, proposed owner roles, supplemental materials | Reconcile the dashboard against the actual candidate and deployment |
| Shared green checks | The 20 common rules, procedures, expected results, and evidence | Establish the standard acceptance contract |
| Validation tests | 320 rows: 20 checks × 16 domains | Bind each applicable row to fixtures, real runners, retained results, and reviewers |
| Domain scenario index | Six specialized scenarios per domain; 96 total | Expand into repository-bound UI/API/database tests |
| Approval artifacts | 16 artifact categories × 16 domains; 256 requirements | Assign named signers and attach accepted evidence |
| Feature-flag rollback | 28 enumerated flags, stop mechanisms, triggers, safe rollback | Verify enforcement and conduct operated drills |

The domain scenario index is not an executable test suite. It names the additional cases that must be implemented and run.

The source baseline also remains historical: a hosted page, a test-file name, or a repository implementation classification does not establish current production readiness. The project's authority document explicitly requires code, tests, and release evidence before "Native and verified."[1]

## 2. Python evidence-gate script

This is the script supplied in the package:

```
"""Offline consistency checker only, not a production authorization service.
Load trusted evidence via existing Semester governance before production use.
"""
from dataclasses import dataclass
from datetime import datetime


@dataclass(frozen=True)
class Target:
    sha: str
    deployment: str
    config: str
    schema: str
    scope: str
    operation: str


@dataclass(frozen=True)
class Evidence:
    gate: str
    target: Target
    passed: bool
```

```
    accepted: bool
    issued_at: datetime
    expires_at: datetime
    evidence_ref: str
    approver: str
    withdrawn: bool = False


def evaluate(target, required, records, now, killed=False):
    reasons = []

    if killed:
        reasons.append("kill_switch")

    if not isinstance(target, Target) or not all(vars(target).values()):
        reasons.append("unresolved_target")

    if not required or len(set(required)) != len(required):
        reasons.append("invalid_gate_contract")

    for gate in required:
        matches = [
            r
            for r in records
            if r.gate == gate and r.target == target
        ]

        if len(matches) != 1:
            reasons.append(gate + ":missing_or_ambiguous")
            continue

        r = matches[^0]

        try:
            current = r.issued_at <= now < r.expires_at
        except TypeError:
            current = False

        if (
            not r.passed
            or not r.accepted
            or r.withdrawn
            or not current
            or not r.evidence_ref
            or not r.approver
        ):
            reasons.append(gate + ":invalid")

    return {
        "allowed": not reasons,
        "reasons": reasons,
    }
```

### What it checks

The helper allows a decision only when every required gate has exactly one matching, current, passed, accepted evidence record.

Matching includes:

```
Commit SHA
+ deployment
+ configuration
+ schema
+ security/cohort scope
+ operation
```

Evidence for “Tenant A registration submission” cannot authorize “Tenant B grade release.”

### Example invocation

```
from datetime import datetime, timezone

target = Target(
    sha="approved-commit-sha",
    deployment="approved-deployment-id",
    config="approved-config-fingerprint",
    schema="approved-schema-fingerprint",
    scope="tenant-a:approved-cohort",
    operation="registration.submit",
)

decision = evaluate(
    target=target,
    required=[
        "security",
        "accessibility",
        "customer_acceptance",
        "rollback",
        "support",
    ],
    records=[],  # Load from a trusted evidence source.
    now=datetime.now(timezone.utc),
    killed=False,
)

# Denied because required evidence is missing.
```

The example values are placeholders, not real deployment identifiers.

### What it does not establish

This helper does not:

- Authenticate a caller.
- Verify a signer's identity or authority.
- Verify report signatures or evidence integrity.
- Determine which gates are required.
- Enforce live tenant membership.
- Resolve policy hierarchy.
- Provide revocation notifications.
- Prevent a race between acceptance checking and command execution.
- Authorize institutional operations on its own.

For production, extend Semester's existing release, activation, flag, and policy evaluators. Do not create a second independent authority system.

### Local checks executed

The 19 assertions covered:

- One valid acceptance.
- Missing evidence.
- Engaged kill switch.
- Withdrawn evidence.
- Expired evidence.
- Future-dated evidence.
- Unaccepted evidence.
- Failed evidence.
- Duplicate evidence.
- Empty gate contract.
- Missing approver.
- Incompatible naive/aware timestamps.
- Unresolved deployment.
- Six target mismatches: SHA, deployment, configuration, schema, scope, and operation.

All passed locally. The supplied test-results document records that limited scope.

## 3. Walkthrough of the 20 shared green-status rules

Every rule needs a real runner or review procedure, a candidate-bound result, and an authorized acceptance record. A missing or skipped test must not count as passed.

### Designed: rules 1–2

| Rule | Check | Passing result | Evidence |
|---|---|---|---|
| 1 — `G-DES-01` | Compare required roles and journeys with accepted scope | Every required action and state is covered; omissions remain held | Journey/state matrix |
| 2 — `G-DES-02` | Review keyboard, focus, errors, responsive behavior, and consequence copy | Accessible interactions and consequential-action semantics are accepted | Design review |

“Designed” means more than a screenshot. It includes loading, empty, denied, stale, offline, conflict, cancellation, and recovery states.

For registration, for example, the design must explain the difference between preview, submitted, source-confirmed, waitlisted, uncertain, and failed—not render every response as “Success.”

### Implemented: rules 3–5

| Rule | Check | Passing result | Evidence |
|---|---|---|---|
| 3 — `G-IMP-01` | Trace mutations from UI through authorization to durable state | Runtime-connected implementation, receipts, and audit; no required mock substitute | Route/service trace |
| 4 — `G-IMP-02` | Forge tenant, owner, identity, and scope at every entry point | Denial without protected metadata or mutation | Authorization-negative report |
| 5 — `G-IMP-03` | Restart, refresh, and retrieve committed entities across devices | Canonical state, version, and provenance survive | Persistence report |

These checks must cover direct APIs, RPCs, functions, workers, storage, search, and realtime—not only the visible UI.

Local storage can be a deliberate personal/offline architecture. It cannot silently substitute for a promised institutional record or cross-device durable workflow.

### Tested: rules 6–8

| Rule | Check | Passing result | Evidence |
|---|---|---|---|
| 6 — `G-TST-01` | Run all applicable cases on the exact candidate and fixtures | Every blocking case passes; no unexecuted case is credited | Run IDs and logs |
| 7 — `G-TST-02` | Qualified assistive-technology, keyboard, and reflow review | Applicable criteria accepted after remediation/retest | Accessibility report |
| 8 — `G-TST-03` | Independent security assessment and blocker rescan | No open launch-blocking finding | Assessor report |

The project's authority gates explicitly include accessibility, security, privacy, and tenant isolation. A functional test suite alone cannot satisfy all of them.[1]

### Deployed: rules 9–11

| Rule | Check | Passing result | Evidence |
|---|---|---|---|
| 9 — G-DEP-01 | Read actual build, commit, configuration, and schema | Match the approved candidate and target | Deployment manifest |
| 10 — G-DEP-02 | Run allowed and denied journeys on the target | Persistence, denials, callbacks, and queues work | Target smoke report |
| 11 — G-DEP-03 | Test old/new application and functions with expanded database | Planned previous-version rollback is compatible | Compatibility report |

A green CI run proves something about the tested build. It does not prove that the intended environment is serving that build with the correct database and policies.

### Institution-authorized: rules 12–14

| Rule | Check | Passing result | Evidence |
|---|---|---|---|
| 12 — G-AUT-01 | Validate approvals, reviewers, target, cohort, data, and operations | Current, scope-matched authority | Approval references |
| 13 — G-AUT-02 | Withdraw or expire approval; attempt activation | Fail closed without broadening scope | Gate-negative report |
| 14 — G-AUT-03 | Run customer UAT and inspect acceptance | Authorized reviewer accepts exact scope and limitations | Customer UAT record |

A signed agreement does not automatically authorize every connector, official write, AI use, or student cohort.

Where institutional approval genuinely does not apply, record the reason and its acceptance. Do not mark it green by inventing approval.

### Production-enabled: rules 15–20

| Rule | Check | Passing result | Evidence |
|---|---|---|---|
| 15 — G-PRD-01 | Attempt use outside approved tenant/cohort/operation | Server, API, and worker deny it | Eligibility report |
| 16 — G-PRD-02 | Kill active high-risk feature; attempt dispatch | Dispatch stops within accepted measured bound | Kill drill |
| 17 — G-PRD-03 | Restore isolated database, matching objects, and configuration | Integrity/access/read checks pass; recovery measured | Recovery record |
| 18 — G-PRD-04 | Exercise unavailable-primary and backup response | Accepted acknowledgement/escalation policy met | Operated drill |

| Rule | Check | Passing result | Evidence |
|---|---|---|---|
| 19 — G-PRD-05 | Export, delete, revoke, and offboard, including holds | Complete policy-correct result | Privacy drill |
| 20 — G-PRD-06 | Observe operating window and reconcile outcome denominators | Accepted service/outcome/stop criteria met | Pilot closeout |

The project's designed authority gates include support readiness, rollback, exportability, commercial readiness, and outcome proof. Some domains also require term-long or multi-term observation; code completion does not erase those requirements. [1]

### 4. Domain approval runbook

Use the same approval sequence for every domain while adding its domain-specific requirements.

```
Accepted scope
    ↓
Pinned candidate and actual-target manifest
    ↓
Implementation and shared-foundation closure
    ↓
Executed tests and qualified review
    ↓
Dark deployment and actual-target verification
    ↓
Applicable professional/customer acceptance
    ↓
Approved tenant, cohort, and operation
    ↓
Operated observation and reconciliation
    ↓
Accepted green cells
```

For each domain, assign four distinct responsibilities:

1. Accountable owner: responsible for completion.
2. Implementation owner: performs the technical or operational work.
3. Acceptance authority: can accept the evidence and authorize the relevant scope.
4. Backup operator: handles service continuity when the primary is unavailable.

The CSVs currently contain proposed roles, not appointed people.

## Verification-script mapping

| Verification area | Required execution |
|---|---|
| Account lifecycle | Candidate-bound account sync, export, erasure, session and membership tests |
| Database security | RLS, grants, security-definer, runtime-role, and cross-tenant tests |

| Verification area | Required execution |
|---|---|
| Domain behavior | UI/API/database assertions using approved deterministic fixtures |
| Integrations | Provider contracts, retries, duplicates, uncertainty, reconciliation |
| Accessibility | Automated checks plus qualified complete-journey evaluation |
| Release | Actual build/config/schema inspection and target smoke |
| Rollback | Previous-version compatibility and operated kill/restore drills |
| Approval | Trusted acceptance records and withdrawn/expired/scope-mismatch tests |

Existing script names must be recovered from the repository and bound to actual runs. The package does not invent shell commands for unverified application interfaces.

## 5. Deployment and feature-flag rollback controls

The rollback matrix treats a feature flag as a control—not proof that the feature is implemented or approved.

### Required stop triggers

Hold the affected operation for:

- Unauthorized disclosure or cross-tenant access.
- Unauthorized official or financial mutation.
- Identity disclosure.
- Data corruption.
- Failed authoritative reconciliation.
- Withdrawn or expired authority.
- Missing operational coverage.
- Breached accepted service thresholds.
- An AI policy bypass.

### Rollback order

```
Declare incident scope
    → engage server kill
    → verify direct API and worker denial
    → pause/checkpoint affected queues
    → reconcile in-flight uncertain outcomes
    → disable scoped feature and invalidate policy caches
    → restore compatible application/functions/workers
    → repair or restore data separately
    → run regression and privacy checks
    → obtain fresh acceptance
    → reactivate bounded scope
```

Key safeguards:

- Do not delete pending transaction evidence.
- Do not blindly replay irreversible commands.
- Do not disable legitimate refunds/cancellations indiscriminately.
- Do not assume an application rollback reverses schema or external records.
- Do not assume an old application build honors current policy without testing.
- Do not expose pseudonymous identities as a rollback shortcut.
- Do not return an authoritative Core module to an unreconciled external source.

The retrieved source enumeration is 28 flags, while the screenshot baseline says 21. That discrepancy remains a reconciliation task, not a completed live-flag audit.

### 6. Material-artifact approval checklist

Each domain must supply and accept the following 16 categories:

1. Product model.
2. User screens.
3. Data model.
4. Workflows.
5. Policy controls.
6. Roles and capabilities.
7. Audit.
8. Support process.
9. Monitoring and SLOs.
10. Documentation.
11. Test plan.
12. Release gate.
13. Migration and rollback plan.
14. Commercial model.
15. Customer-success model.
16. Operating owner.

These categories come directly from the project's native authority-gate document. The exported matrix adds required contents, proposed sign-off role, named signer, evidence reference, and acceptance status for all 16 domains.[1]

A template is not an accepted artifact. An artifact is not accepted until the appropriate reviewer evaluates its evidence for the exact release scope.

### 7. What remains before any domain can turn green

The master report keeps the principal shared blockers visible:

- Candidate and actual-target uncertainty.
- Incomplete all-entry-point authorization/isolation.
- Local/blob/duplicate data-authority paths.
- Unproved adapters and asynchronous reconciliation.
- AI bypasses, broker enforcement, and evaluation gates.
- Qualified accessibility acceptance.
- Staffed support and paging.
- Operated recovery, rollback, and privacy evidence.
- Institutional/commercial authority.
- Stale inventories and conflicting claims.

The package is ready to serve as the execution register. Its statuses must advance only when actual implementation, tests, target evidence, and authorized sign-offs replace the current placeholders.

⁂

1. The Main Semester design system (2) copy/docs/native-platform/NATIVE_AUTHORITY_GATES.md